Finn Fassbender
Ahmet Alperen
Gngr

1	2	Σ

Assignment Sheet Nr. 12

Question 1: Computational costs of convolutions

Suppose in a neural network, we have two tensors A and B with sizes $3 \times 32 \times 32$ and $16 \times 32 \times 32$ (channel, height, width). Include the necessary padding, ignore the bias tensor and batch size.

(a) Compute the number of multiplication operations if we convolve a kernel of spatial size 5×5 so that $B = \texttt{conv}(A)$ (with a stride of 1).

$$5 \times 5 \cdot (3 \cdot 16) \times 32 \times 32 = 1228800$$

(b) Compute the number of multiplication operations if we convolve two kernels each of spatial size 3×3 so that $H = \texttt{conv}_1(A)$, $B = \texttt{conv}_2(H)$ where H has n channels.

$$3 \times 3 \cdot ((3 \cdot n) + (n \cdot 16)) \times 32 \times 32 = 175104n$$

(c) Choose n as large as possible so that (b) requires less multiplication operations than (a).

$$n = \lfloor 1228800/174104 \rfloor = 7$$

(d) Regular convolutions might have redundancy in their weights, so it is often more efficient to factorize spatial and cross-channel into two steps (*depthwise separable convolutions*). For this, we first convolve the kernel of (a) without any cross-channel computations. That means, that each channel is convolved with an individual kernel, resulting in one output channel per input channel. This leads to a tensor of size $3 \times 32 \times 32$. Afterwards, we apply a regular kernel of spatial size 1×1. The final output results in B.
$H = \texttt{conv}_{spatial}(A)$, $B = \texttt{conv}_{channels}(H)$

$$5 \times 5 \cdot 3 \times 32 \times 32 + 1 \times 1 \cdot 3 \cdot 16 \times 32 \times 32 = 125952$$

(e) Compute the number of multiplication operations of (a) and (d), comment on the results.

Separating spatial and cross-channel computation (depthwise + pointwise) gives a nearly 10-fold reduction in multiplications. Replacing a single wide kernel by two smaller spatial kernels (two 3×3) can approximate spatial 5×5 at lower cost only if the intermediate channel count n is kept small ($n \leq 7$).

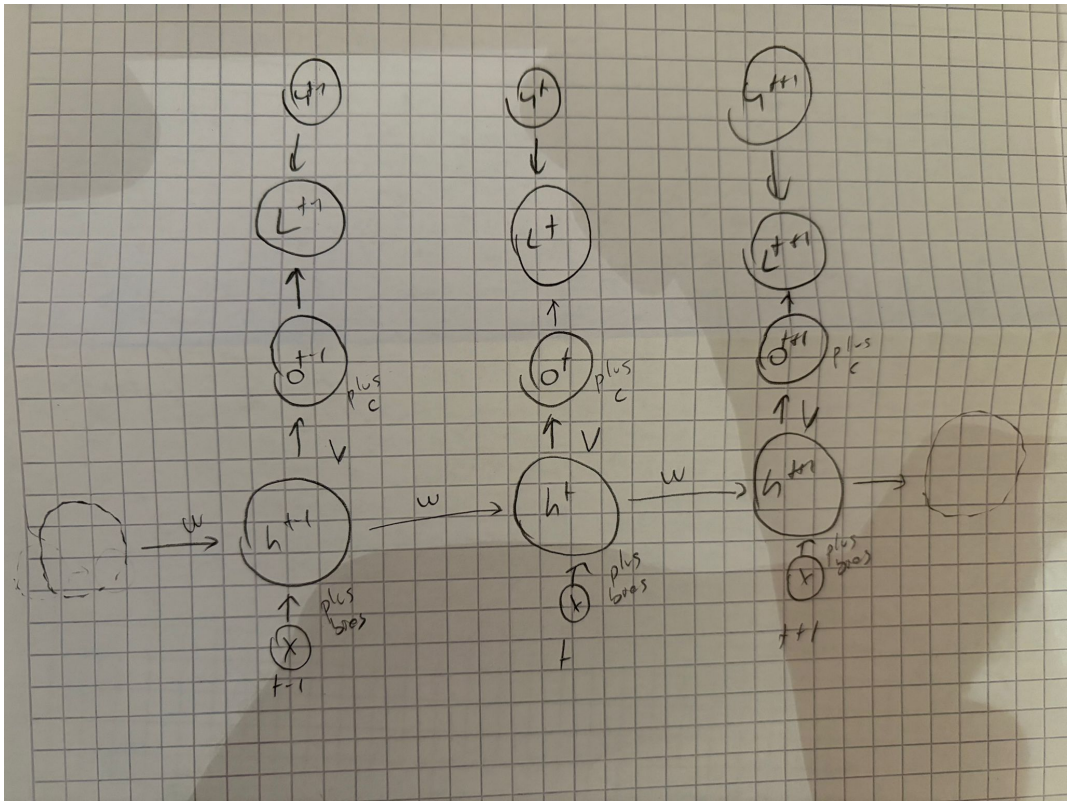
Question 2: Recurrent Neural Networks and Transformer

(a) What is the key idea of *self-attention* in a Transformer, compared to recurrent networks?

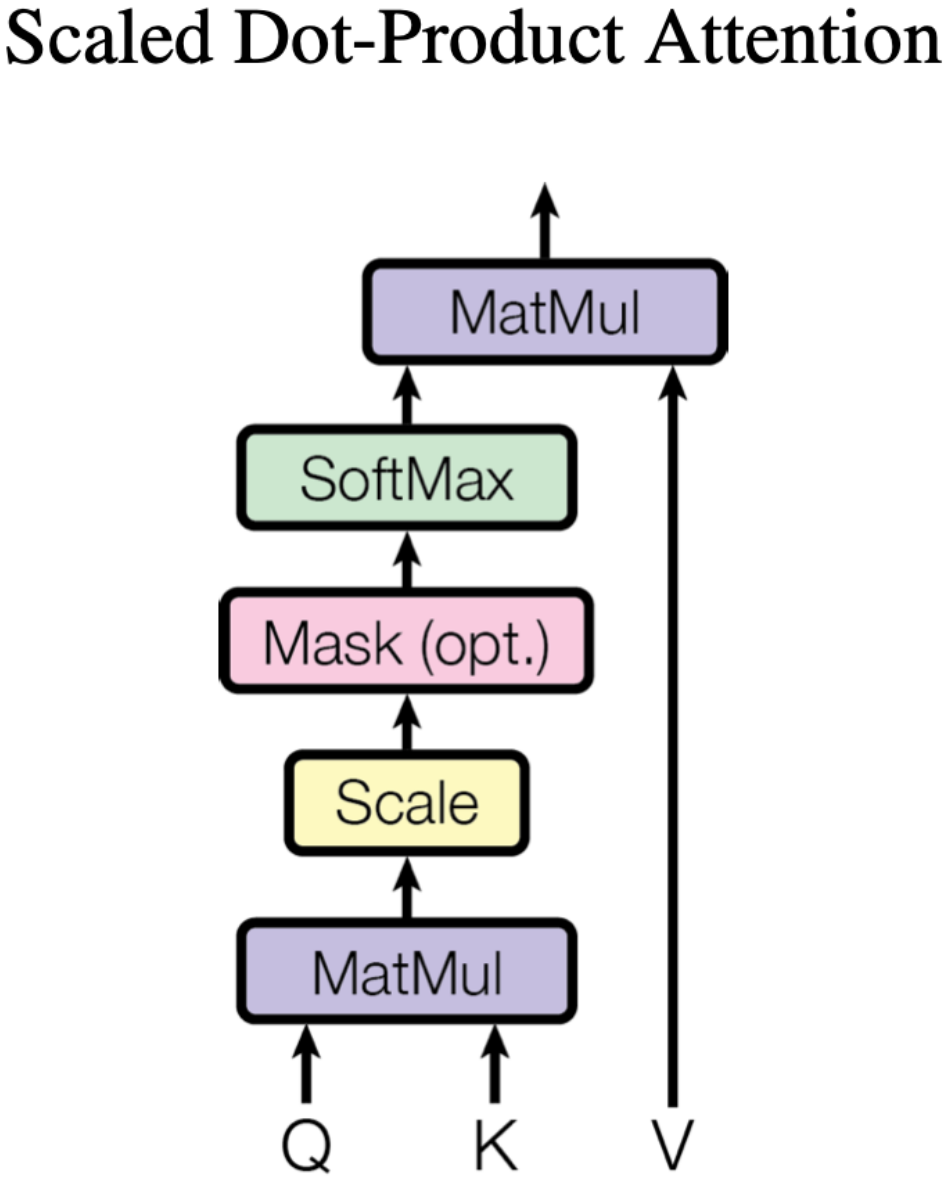
Self-attention lets every token directly read and weight every other token in the sequence parallely (via learned query/key/value projections and softmax weighting), whereas an RNN passes information along using a sequential hidden state.

(b) Given the following equations, draw the computational graph to compute the training loss of the corresponding recurrent neural network and unfold it for the time steps $t-1$, t and $t+1$.

$$
\begin{aligned}
a^{(t)} &= b + Wh^{(t-1)} + Ux^{(t)} \\
h^{(t)} &= \tanh\left(a^{(t)}\right) \\
o^{(t)} &= c + Vh^{(t)} \\
\hat{y}^{(t)} &= \mathrm{softmax}(o^{(t)})
\end{aligned}
$$



(c) Write down the corresponding names next of the five white nodes and complete the equation for the self-attention.



$$\texttt{Attention}(Q, K, V) = \texttt{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$